Validating distillation performance by Monte Carlo: a guide to `qmsd.mc_distill` and the next steps

prime_msd project notes for Aanha

28 October 2026

1 Where this fits

The paper we are assembling has three parts: new magic-state distillation codes (the $\gamma < 1$ codes at $p = 7, 17, 19$ and the Reed–Solomon ladder to $p = 97$), a structural result (no small $\gamma < 1$ code for $p \leq 13$ in the Reed–Muller framework), and an *exact performance engine* that assigns every catalog code its output error curve, threshold and distillation cost. Every cost and threshold claim in the paper rests on that engine. Nobody has yet checked it against an independent simulation, and nobody in the literature has published a noisy-circuit simulation of any qudit triorthogonal code. That is the gap you are filling, in two steps:

1. **Channel-level Monte Carlo** (this document): sample the noise model the engine assumes, run the post-selection exactly as the engine defines it, and show that the sampled output error and acceptance probability land on the engine's exact curves. Deliverable: one figure like Figure 1 with your own points on it, for at least four codes, plus a short table.
2. **Circuit-level simulation in `sdim`**: replace the idealised channel by a noisy encoding-and-measurement circuit for one or two small codes and measure how far the real output error departs from the channel-level curve. Deliverable: a notebook and a plot of simulated against predicted performance.

Step 1 is deliberately easy. The sampler is 44 lines, the exact curves are already computed and stored, and the codes involved have at most 44 qudits. Its value is that it validates everything downstream, and that every piece of it (the noise model, the acceptance test, the two error notions, the statistics) reappears in step 2.

2 The model the engine computes

A triorthogonal code is specified for our purposes by two integer matrices over $\mathbb{F}_p$: the stabiliser rows G_0 (the shortened generator; call its row span A) and the k canonical logical rows H_1 (one per output). Both come straight out of the catalog, see Appendix A.

Noise. Each of the n input magic states independently suffers a Z^{j_l} error with $j_l \neq 0$ with probability ε, the nonzero value uniform on $\mathbb{F}_p^\times$. This is *twirled depolarising noise* in the convention of arXiv:2610.08012; the repo's older δ is $\delta = \frac{p}{p-1}\varepsilon$. The whole error is a vector $j \in \mathbb{F}_p^n$.

Acceptance. The protocol measures the stabilisers and keeps the round iff every syndrome is trivial: $G_0\, j = 0$, i.e. $j \in A^\perp$. The acceptance probability is $P_{\mathrm{acc}}(\varepsilon) = \Pr[G_0 j = 0]$.

Failure. An accepted round delivers a wrong output i iff the logical row sees the error: $(H_1 j)_i \neq 0$. Two error notions matter:

$$\varepsilon_{\text{out}}^{\text{avg}}(\varepsilon) = \frac{1}{k}\sum_{i=1}^{k} \Pr[(H_1 j)_i \neq 0 \mid \text{accept}] \qquad \text{(per output; what a recursion consumes)},$$
$$\varepsilon_{\text{out}}^{\text{blk}}(\varepsilon) = \Pr[H_1 j \neq 0 \mid \text{accept}] \qquad \text{(block: any output wrong; the paper's formula)}.$$

For $k = 1$ they coincide; for $k > 1$ the block error is between $1\times$ and $k\times$ the per-output error, and tends to $k\times$ as $\varepsilon \to 0$ because low-weight errors rarely hit two outputs at once.

Leading order and threshold. For small ε, $\varepsilon_{\text{out}} \approx c\varepsilon^d$ where d is the code distance and $c = L_d/(p-1)^d$ with L_d the number of weight-d accepted logical errors. The *threshold* ε^* is the smallest root of $\varepsilon_{\text{out}}(\varepsilon) = \varepsilon$: above it the round makes things worse. The overhead exponent $\gamma = \ln(n/k)/\ln d$ is the asymptotic scaling of raw states per output when the code is used recursively.

How the engine gets exact numbers. By MacWilliams duality, $\Pr[j \in C^\perp] = |C|^{-1}\sum_{c\in C}(1-\eta)^{\text{wt}(c)}$ with $\eta = p\varepsilon/(p-1)$, so it only has to enumerate the *small* space A ($p^{\dim A}$ words; $\dim A = 2, 5, 3$ for the three codes below) and the cosets $A + h_i$. Those weight enumerators are integers; the engine combines them into one polynomial and evaluates it in `mpmath`, which is why the curves in Figure 1 go down to 10^{-11} without noise. Your sampler has no such trick, and seeing where it loses the curve is one of the exercises.

3 The codes and their reference numbers

Use these three first; they are small, exact, and span $p = 3, 7, 23$.

code	catalog id / builder	n	k	$\dim A$	d	c_ε	ε^*
$[[8,1,2]]_3$	RM(1, 2) punctured at column 1 (App. A)	8	1	2	2	2.000	0.2110
$[[44,5,3]]_7$	`imported-p7-n44-k5-*` (any of the ten)	44	5	5	3	1.111	0.1732
$[[18,5,4]]_{23}$	`imported-p23-n18-k5-4cc848da1772`	18	5	3	4	0.2874	0.3963

Exact per-output values from the engine (these are the numbers your points must reproduce within their error bars):

		$\varepsilon = 0.2$	0.1	0.05	0.02	0.01
$[[8,1,2]]_3$	P_{acc}	0.2157	0.4464	0.6679	0.8515	0.9229
	ε_{out}	1.85×10^{-1}	3.24×10^{-2}	6.40×10^{-3}	8.84×10^{-4}	2.10×10^{-4}
$[[44,5,3]]_7$	P_{acc}	1.16×10^{-4}	9.78×10^{-3}	0.1047	0.4111	0.6426
	ε_{out}	4.42×10^{-1}	4.87×10^{-3}	2.46×10^{-4}	1.08×10^{-5}	1.22×10^{-6}
$[[18,5,4]]_{23}$	P_{acc}	1.81×10^{-2}	0.1501	0.3972	0.6951	0.8345
	ε_{out}	2.34×10^{-3}	5.83×10^{-5}	2.51×10^{-6}	5.23×10^{-8}	3.06×10^{-9}

4 Inside `qmsd/mc_distill.py`

The whole sampler is one function. Read it next to Section 2; every line implements one sentence there.

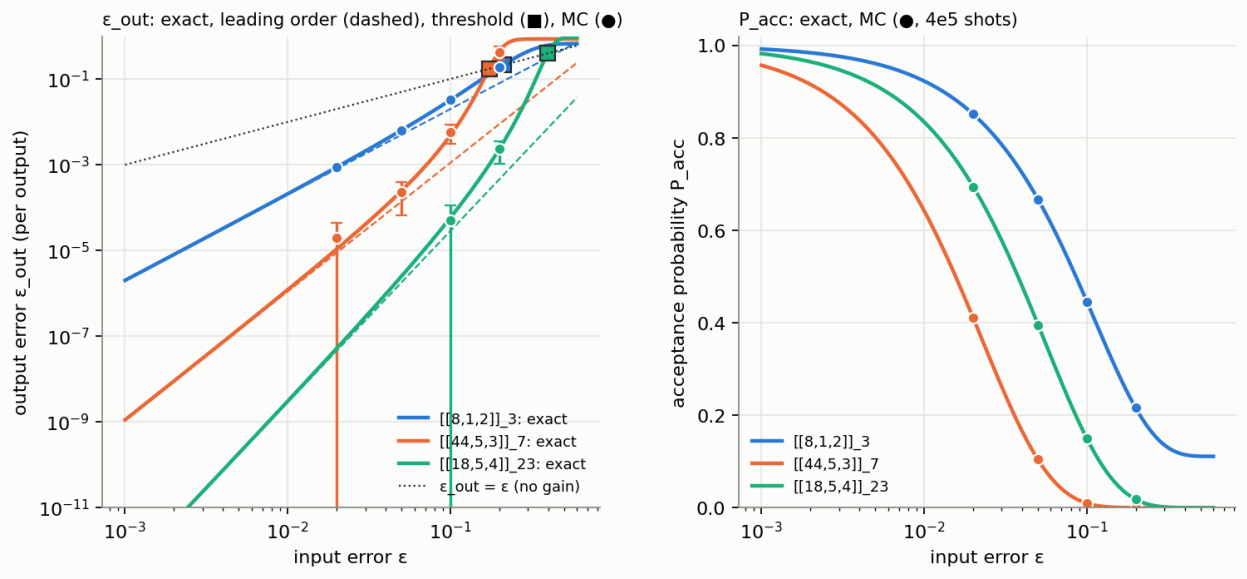


Figure 1: Left: per-output error against input error for the three codes. Solid: exact engine. Dashed: leading order $c\varepsilon^d$. Square: threshold $\varepsilon_{\text{out}} = \varepsilon$. Dots with error bars: `mc_distill.simulate` with 4×10^5 shots at $\varepsilon \in \{0.2, 0.1, 0.05, 0.02\}$. Right: acceptance probability, same conventions. Your first deliverable is this figure with your own dots on it.

```python
def simulate(G0, H1, p, eps, shots, seed=0, batch=200_000) -> dict:
    G0 = np.asarray(G0, dtype=np.int64) % p # stabiliser rows, shape (dim A, n)
    H1 = np.asarray(H1, dtype=np.int64) % p # logical rows, shape (k, n)
    k, n = H1.shape
    rng = np.random.default_rng(seed)
    acc = hits = blk = 0
    done = 0
    while done < shots:
        b = min(batch, shots - done)
        # (1) the noise: each coordinate is nonzero w.p. eps, value uniform on 1..p-1
        j = np.where(rng.random((b, n)) < eps, rng.integers(1, p, size=(b, n)), 0)
        # (2) acceptance: every syndrome G0 j (mod p) must vanish
        ok = ~np.any((j @ G0.T) % p, axis=1)
        # (3) failure: for accepted shots, which outputs see the error?
        logical = ((j[ok] @ H1.T) % p) != 0 # shape (accepted, k), boolean
        acc += int(ok.sum())
        hits += int(logical.sum()) # total wrong outputs
        blk += int(logical.any(axis=1).sum()) # shots with any wrong output
        done += b
    def frac(x, m): # binomial estimate with 1-sigma error
        f = x / m if m else float("nan")
        return f, math.sqrt(f * (1 - f) / m) if m else float("nan")
    pa, pa_err = frac(acc, shots) # P_acc
    eb, eb_err = frac(blk, acc) # block error, conditioned on acceptance
    ea = hits / (k * acc) if acc else float("nan") # per-output average
    ea_err = math.sqrt(max(ea * (1 - ea), 0.0) / acc) * math.sqrt(k) if acc else float("
        nan")
    return {...}
```

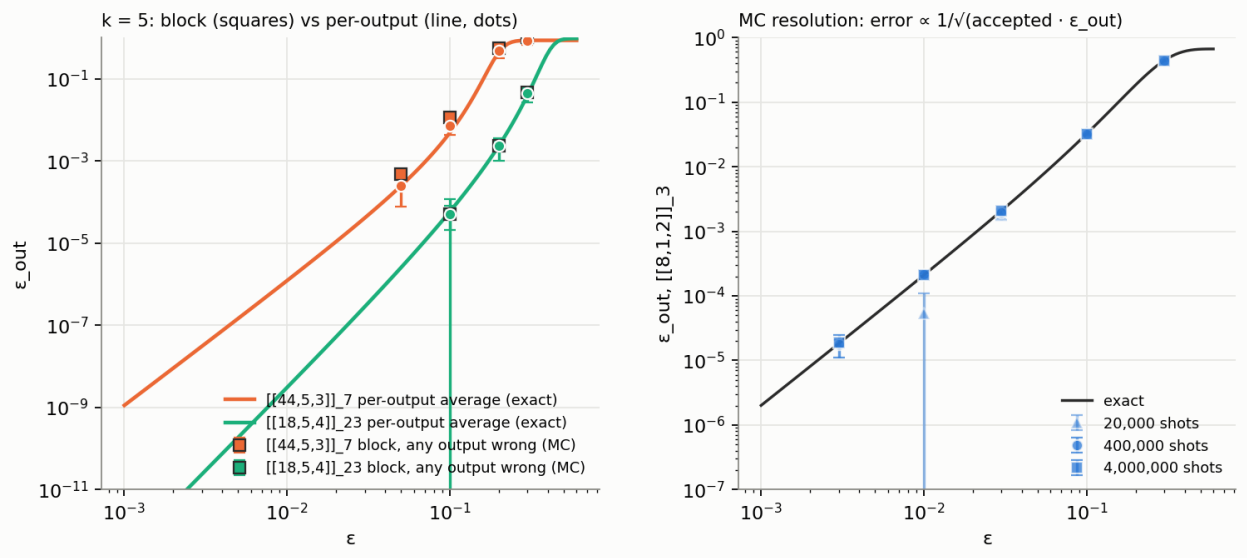


Figure 2: Left: for the two $k = 5$ codes, the block error (squares, Monte Carlo) sits above the per-output average (line exact, dots Monte Carlo) and the ratio grows toward $k = 5$ as ε falls. Right: the sampler's resolution for $[[8, 1, 2]]_3$ at 2×10^4, 4×10^5 and 4×10^6 shots; the error bar scales as $1/\sqrt{(\text{accepted shots}) \cdot \varepsilon_{\text{out}}}$, so each decade of ε_{out} costs a decade of shots.

Things worth noticing:

- **Everything is a matrix product mod p.** Acceptance and failure are linear tests on the integer vector j. There is no quantum state anywhere; that is what makes the channel-level model exact and cheap, and it is exactly what `sdim` will add back.
- **Batching.** Shots are generated 2×10^5 at a time so memory stays at a few tens of MB regardless of `shots`. `seed` makes a run reproducible.
- **The error bars are binomial.** P_{acc} is a fraction of `shots`; the block error is a fraction of *accepted* shots. The per-output average counts wrong outputs across k correlated indicators, so its error is bounded by $\sqrt{k}$ times the block-style estimate rather than computed exactly. (Exercise 5 asks you to do better.)
- **What is not modelled.** No gate noise, no measurement noise, no X errors (they are not post-selected by this code family at all), no correlated noise, and no Clifford correction step. The sampler validates the engine's arithmetic under the engine's own assumptions; the circuit simulation tests the assumptions.

5 Exercises

Work in a scratch directory inside the repo, run everything with `PYTHONPATH=.` from the repo root, and keep the memory cap (`ulimit -v 4000000`) and single-thread BLAS (`OMP_NUM_THREADS=1`) on the shared boxes. Write each result into a small JSON next to your script; the plots come last.

Exercise 1: reproduce one point

Build $[[8,1,2]]_3$ (Appendix A) and run `simulate(G0, H1, 3, 0.05, shots=400_000, seed=1)`.

What to compare against

$P_{\text{acc}} = 0.6679$ and $\varepsilon_{\text{out}} = 6.40 \times 10^{-3}$. A run with seed 1 gave 0.6671 ± 0.0007 and $6.33 \times 10^{-3} \pm 1.5 \times 10^{-4}$. Anything within about 3σ is a pass; a 5σ miss means a bug, not bad luck.

Exercise 2: the whole curve

Repeat for $\varepsilon \in \{0.3, 0.2, 0.1, 0.05, 0.02, 0.01\}$ and all three codes, and plot the points over the exact curves (the exact curve is three lines of code, Appendix B). Use a log–log axis.

What to compare against

Figure 1. At $\varepsilon = 0.02$ and 0.01 the $[[18,5,4]]_{23}$ points will be zero with a zero error bar: 4×10^5 shots cannot see $\varepsilon_{\text{out}} \sim 10^{-8}$. That is correct behaviour, and the next exercise quantifies it.

Exercise 3: resolution and cost

Show empirically that the relative error of ε_{out} scales as $1/\sqrt{N_{\text{acc}}\,\varepsilon_{\text{out}}}$ by running $[[8,1,2]]_3$ at $\varepsilon = 0.03$ with 2×10^4, 2×10^5, 2×10^6 shots. Then estimate how many shots a 10% measurement of $[[18,5,4]]_{23}$ at $\varepsilon = 0.01$ would need.

What to compare against

Right panel of Figure 2. For the estimate: $P_{\text{acc}} = 0.83$ and $\varepsilon_{\text{out}} = 3.1 \times 10^{-9}$, so 10% needs about 100 failures, i.e. 3×10^{10} accepted and 4×10^{10} shots. At $\sim 10^6$ shots per second that is half a day for one point. This is why the engine exists, and why Exercise 5 matters.

Exercise 4: distance and leading coefficient from data

From your points at $\varepsilon \in \{0.1, 0.05, 0.02\}$ fit $\log \varepsilon_{\text{out}}$ against $\log \varepsilon$ for $[[44,5,3]]_7$. The slope is the distance, the intercept the coefficient.

What to compare against

Slope ≈ 3 (exactly $d = 3$ only as $\varepsilon \to 0$; between 0.1 and 0.02 you should see 2.8 to 3.0), coefficient ≈ 1.1. The exact values are $d = 3$, $c_\varepsilon = 1.111 = L_3/6^3$ with $L_3 = 240$ weight-3 accepted logical errors per output.

Exercise 5: threshold by bisection

Find ε^* for $[[8, 1, 2]]_3$ as the root of $\varepsilon_{\text{out}}(\varepsilon) - \varepsilon$, using the sampler at 10^5 shots per evaluation. Think about why bisection needs the sign of the difference and how many shots that requires near the root.

What to compare against

$\varepsilon^* = 0.2110$ (engine). The paper arXiv:2610.08012 Table 2 prints 0.211. Expect ± 0.003 from the sampler.

Exercise 6: block against per-output

For $[[44, 5, 3]]_7$ at $\varepsilon \in \{0.3, 0.1, 0.05\}$ compute the ratio $\varepsilon_{\text{out}}^{\text{blk}} / \varepsilon_{\text{out}}^{\text{avg}}$ from the sampler's two estimates. Explain the trend.

What to compare against

Left panel of Figure 2: the ratio is about 1.3 at $\varepsilon = 0.3$ and about 2.1 at $\varepsilon = 0.05$, heading to $k = 5$. At high ε the accepted errors are heavy and hit several outputs at once; at low ε a weight-3 error hits one output.

Exercise 7: better error bars

The per-output error bar in `simulate` is a bound, not an estimate. Modify a copy of the function to accumulate, per shot, the number of wrong outputs $w \in \{0, \ldots, k\}$, and compute the variance of w/k directly. Compare the new bar with the old one at $\varepsilon = 0.05$ for $[[44, 5, 3]]_7$.

What to compare against

The new bar must be smaller than or equal to the old one and never below the block-style bar divided by $\sqrt{k}$. Report both.

Exercise 8: the weight histogram and MacWilliams

Modify a copy of `simulate` to histogram the Hamming weight of every *accepted* j and of every accepted j that fails output 1. Compare the low-weight bins, divided by the number of shots, against the engine's exact counts: the number of accepted errors of weight w is B_w and the probability of one of them is $(\varepsilon/(p-1))^w (1-\varepsilon)^{n-w}$.

What to compare against

For $[[8, 1, 2]]_3$: $L_2 = c_\varepsilon (p-1)^d = 2 \cdot 4 = 8$ weight-2 accepted logical errors. Your weight-2 failing-bin frequency at $\varepsilon = 0.05$ must be $8\,(0.025)^2 (0.95)^6 \approx 3.7 \times 10^{-3}$ of all shots, within error.

Exercise 9: importance sampling

Exercise 3 showed that direct sampling cannot reach small ε. Write a sampler that draws the error *weight* w from a distribution of your choice, then a uniformly random pattern of that weight (positions and nonzero values), and reweights each shot by the true probability $\binom{n}{w}(\varepsilon/(p-1))^w(1-\varepsilon)^{n-w}$ divided by your proposal. Use it to measure $[[18,5,4]]_{23}$ at $\varepsilon = 0.01$ and 0.001.

What to compare against

3.06×10^{-9} at $\varepsilon = 0.01$. At $\varepsilon = 0.001$ compute the exact value yourself with Appendix B; it is close to $c_\varepsilon\, \varepsilon^4 = 2.9 \times 10^{-13}$. With 10^6 reweighted shots concentrated on $w \in \{4,5,6\}$ you should get both to a few percent.

Exercise 10: your own codes

Run Exercise 2 on your $[[215,28,4]]_3$ and $[[689,40,5]]_3$ entries. The exact engine may refuse the larger one (too many words to enumerate; it says so). Then the sampler is the only curve we have, and your points become the reference.

What to compare against

Where the engine runs, agreement as before. Where it does not, check your points against the leading order $A_d/(p-1)^d \cdot \varepsilon^d$ from the catalog's A_d at the two smallest ε you can resolve, and say in your write-up which regime you are in.

6 Toward the circuit simulation

The channel model assumes that the only noise is an independent Z error on each input magic state. A real round also has: noisy Clifford gates in the encoder, noisy syndrome measurements, idle errors, and X-type errors that this family does not post-select on. The question for `sdim` is how much each of these shifts the curve, and in which direction. Before writing any circuit, write down what you expect:

- A gate error rate p_g adds terms to ε_{out} that are *first order* in p_g, independent of d. On a log–log plot against ε the curve flattens into a floor at small ε; the floor height is the number. Predict it from the gate count before you simulate it.
- Measurement errors change P_{acc} more than ε_{out}: a wrong syndrome mostly rejects good rounds.
- The channel-level curve is the $p_g \to 0$ limit. Your first `sdim` run should reproduce Exercise 1 with all circuit noise switched off; if it does not, the circuit is wrong, not the engine.

Start with $[[8,1,2]]_3$ (8 qutrits, two stabilisers, one logical) and then $[[18,5,4]]_{23}$ if `sdim` supports $p = 23$; $[[44,5,3]]_7$ is the realistic target. The encoding circuit comes from G_0 and H_1 by the standard CSS-encoder construction; ask before building it so we agree on the gate set.

7 Deliverables and checklist

1. `runs/mc/<code>_<eps>.json` per point (shots, seed, all four estimates and errors), produced by a script committed under `scratchpad/` or `docs/`.
2. Figure 1 redrawn with your points, at least four codes, the three above plus one of yours.
3. A half-page note: which points agree, the measured scaling of Exercise 3, the fitted d and c of Exercise 4, and the importance-sampling result of Exercise 9.
4. Open a pull request against `main` from your branch. Leave the `primepower_msd/` directory untouched.

A Getting G_0 and H_1

For a catalog record:

```
from qmsd.results import load_result_catalog
from qmsd.performance import record_matrices
wanted = "imported-p23-n18-k5-4cc848da1772"
rec = next(r for r in load_result_catalog() if r.artifact_id == wanted)
G0, H1 = record_matrices(rec) # integer arrays mod p; shapes (dim A, n) and (k, n)
```

For the $k = 1$ Reed–Muller code $[[8, 1, 2]]_3$ (puncture one column of $\mathrm{RM}(1, 2)$ over $\mathbb{F}_3$; any column is equivalent):

```
from qmsd.reedmuller import r_max, rm_generator
from qmsd.triorthogonal import build_triorthogonal_code
from qmsd.performance import canonical_logical_rows
p, m = 3, 2
r = r_max(m, p); G = rm_generator(r, m, p)
G0 = build_triorthogonal_code(p, m, r, [1], G=G)["G0"]
H1 = canonical_logical_rows(G, [0], p)
```

B The exact curve in three lines

```
from qmsd.performance import profile_exact, leading_order, threshold
prof = profile_exact(G0, H1, p, kinds=("avg",)) # p**dim(A) words
print(float(prof.p_accept(0.05)), float(prof.eps_out(0.05, "avg")))
lo = leading_order(prof, "avg"); th = threshold(prof, "avg")
print(lo.d, lo.c_eps, th.eps)
```

The block kind (`kinds=("avg","block")`) costs $p^{\dim A+k}$ words and is refused above 5×10^7; for $k = 5$ at $p = 23$ that is already too many, which is why the block error in Figure 2 comes from the sampler.

C Files

- `qmsd/mc_distill.py` is the sampler and `qmsd/performance.py` the exact engine. The test suite compares them in `tests/test_performance.py` (`test_monte_carlo_agrees`).
- `PERFORMANCE.md`: the engine's write-up and validation against arXiv:2610.08012 Table 2. `COVERAGE.md`: the per-prime table of codes.

- `docs/aanha_mc_guide/make_figures.py`: produced every figure and number in this document; run it to regenerate them.